

# Lab 11 - Captions, Overlays, Safe Areas and Export Profiles

Course: Generative AI for Image and Video Creation (TGS-2020505925) | Version v11.0 | 6 September 2026 Topic 04: Generative AI Video Creation, Animation and Editing Outcome: ELO5 | In-class time: 30 minutes of scheduled practical time

| Competency    | Statement                                                                        |
|---------------|----------------------------------------------------------------------------------|
| Ability A6    | Design and apply video analytics algorithms for high-level video analytics tasks |
| Knowledge K10 | Activity tracking, generative models, scene understanding and event discovery    |

---

## Scenario

The film is approved. Now it has to ship in three aspect ratios with burned-in captions that are legible on a phone, a logo that stays inside the safe area in every crop, and a file size that survives the platform's limit. Build the export ladder and measure every constraint rather than eyeballing it.

## What you will produce

Three exported renditions (16:9, 1:1, 9:16) with burned-in captions and overlay, out/export-report.csv with per-rendition dimensions, duration, file size and bit-rate, and out/safe-area-check.csv proving the caption and logo stay inside the safe area in all three.

Tools: Python 3 | opencv-python | NumPy | no network, no paid service; silent fixture (OpenCV export does not preserve audio)

## Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: python3, opencv-python, numpy.

API currency. Everything taught here runs on the base opencv-python wheel. Two things that appear in older tutorials are *\*not\** in that wheel and are deliberately avoided: cv2.TrackerCSRT\_create / cv2.TrackerKCF\_create and the cv2.quality SSIM module, both of which live in opencv-contrib-python. Where SSIM is needed the course ships its own NumPy implementation.

## Workflow

1. Parse the caption cue list
2. Burn captions with a measured safe area

3. Render the three aspect-ratio renditions
4. Verify safe area and legibility numerically
5. Report size, bit-rate and constraint compliance

## Files in this folder

| Path                          | What it is                                                                                                                           |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| reference/nq-approved-cut.mp4 | Synthetic 12-second 1920x1080 25 fps approved cut (deterministically rendered with OpenCV - SIMULATED, not generative-model output)  |
| reference/nq-logo-overlay.png | 240x120 logo overlay with an alpha channel (deterministically rendered - SIMULATED)                                                  |
| data/captions.json            | The caption cue list: start, end, text and speaker for each cue                                                                      |
| data/export-profiles.json     | The three delivery profiles with target dimensions, safe-area insets, maximum file size and minimum caption H-glyph height in pixels |
| export_ladder.py              | Runnable script - see the steps below                                                                                                |
| PROMPTS.md / PROMPTS.pdf      | The reusable prompt and specification pack                                                                                           |
| out/                          | Everything you produce (created on first run)                                                                                        |

Provenance. Every image and clip in reference/ is either a SIMULATED classroom asset rendered deterministically with OpenCV, an AI-generated reference copied unchanged with a PROVENANCE.md beside it, or a real prerecorded sequence with its source and SHA-256 recorded. Nothing in this folder may be relabelled: a rendered animation is never presented as model output, and a simulated stand-in is never presented as a photograph.

## Steps

1. Read data/captions.json. Each cue has start and end times in seconds and the caption text. Confirm the cues do not overlap and that the last cue ends within the clip duration - this script rejects overlaps instead of silently selecting one cue.

```
python3 -c "import json;c=json.load(open('data/captions.json'))['cues'];print(len(c),'cues, overlaps:',[i for i in range(1,len(c)) if c[i]['start']<c[i-1]['end']] or 'none')"
```

2. Read data/export-profiles.json. Each profile gives the output dimensions, the safe-area inset as a fraction, the maximum file size in bytes, and the minimum caption H-glyph height in pixels. That last one is the legibility constraint: a caption scaled down with the frame becomes unreadable on a phone.
3. Compute the required font scale per profile rather than hard-coding it. Use cv2.getTextSize to measure the rendered height at scale 1.0, then solve for the scale that reaches the profile's minimum H-glyph height.

```
(tw, th), base = cv2.getTextSize('Hg', cv2.FONT_HERSHEY_SIMPLEX, 1.0, 2)
scale = required_cap_height_px / float(th)
```

4. Draw a caption with a background plate rather than plain text. Measure the text, draw a filled rectangle with a few pixels of padding, then draw the text over it. Text without a plate is unreadable over a bright frame and there is no way to fix it later.

```
cv2.rectangle(frame, (x - 8, y - th - 8), (x + tw + 8, y + base + 4), (18, 18, 22), -1)
cv2.putText(frame, text, (x, y), cv2.FONT_HERSHEY_SIMPLEX, scale,
            (245, 245, 245), 2, cv2.LINE_AA)
```

- Define the safe area numerically: inset the frame by the profile's fraction on all four sides. Every caption plate and the logo must lie entirely inside it. Compute the safe rectangle in pixels for each profile and record it.
- Composite the logo using its alpha channel, not a rectangular paste. Read the PNG with `cv2.IMREAD_UNCHANGED` to get four channels, split off the alpha, normalise it to [0,1] and blend.

```
logo = cv2.imread('reference/nq-logo-overlay.png', cv2.IMREAD_UNCHANGED)
bgr, alpha = logo[:, :, :3], logo[:, :, 3:4].astype(np.float32) / 255.0
roi[:] = (alpha * bgr + (1 - alpha) * roi).astype(np.uint8)
```

- Decide the reframing rule for each aspect ratio and write it down before coding it. For 1:1 and 9:16 you must either CROP (loses the sides) or LETTERBOX (keeps everything, adds bars). State which you chose per profile and why - this is a content decision, not a technical default.
- Implement centre-crop-then-scale for the profiles where you chose crop. Compute the source rectangle with the target aspect ratio, crop it, then resize with `cv2.INTER_AREA` - the correct interpolation for downscaling.

```
src_ar = w / h
dst_ar = tw_ / th_
if src_ar > dst_ar:      # source is wider: crop the sides
    new_w = int(h * dst_ar); x0 = (w - new_w) // 2
    crop = frame[:, x0:x0 + new_w]
else:                  # source is taller: crop top and bottom
    new_h = int(w / dst_ar); y0 = (h - new_h) // 2
    crop = frame[y0:y0 + new_h, :]
```

- Implement letterbox for the profiles where you chose it: scale to fit inside the target and pad the remainder with `cv2.copyMakeBorder`. Record the pad size - a 9:16 letterbox of a 16:9 source is mostly bars, which is usually the argument for cropping instead.
- Render all three profiles.

```
python3 export_ladder.py --video reference/nq-approved-cut.mp4 --captions data/captions.
json --profiles data/export-profiles.json --logo reference/nq-logo-overlay.png --out out
```

- Verify the safe area programmatically. For each profile and each caption cue, compute the caption plate rectangle and the logo rectangle in output pixels, and test that both lie inside the safe rectangle. Write `out/safe-area-check.csv` with a PASS/FAIL per cue per profile.
- Verify legibility. Report the achieved caption H-glyph height in pixels for each profile and confirm it meets the profile minimum. Then express it as a percentage of frame height as a layout proxy; inspect on a phone because reading speed, viewing distance and typography also matter.
- Verify contrast. Record the relative-sRGB luminance contrast ratio of the chosen solid text and plate colours, using  $(\text{lighter}+0.05)/(\text{darker}+0.05)$ . This is a design-colour check; inspect decoded frames for compression and antialiasing effects.
- Measure the exports. For each rendition record dimensions, decoded frame count, duration, file size in bytes, and the average container bit-rate in bits per second. Compare each against the profile's maximum file size and mark PASS or FAIL.
- Where a profile fails on size, state the lever you would pull and why - resolution, frame rate, or encoder quality - and note the trade-off each carries. Do not silently re-encode until it fits and report only the passing run.
- Confirm every rendition still carries the correct provenance. If the source cut is a simulated classroom asset, all three exports are too; the label travels with the derivative. Record this explicitly.
- Save the three renditions, `out/export-report.csv` and `out/safe-area-check.csv`, then complete the evidence checklist.

## Verify

Three renditions exist at the profile dimensions with the same duration as the source, out/safe-area-check.csv shows PASS for every cue in every profile, the achieved caption H-glyph height meets each profile's minimum, and out/export-report.csv records size and bit-rate with a PASS/FAIL against each profile's limit.

## Expected outputs

- out/export\_16x9.mp4 at 1920x1080, out/export\_1x1.mp4 at 1080x1080, out/export\_9x16.mp4 at 1080x1920.
- All three the same duration as the source (12.00 s).
- out/safe-area-check.csv - one row per (profile, cue) plus a logo row, all PASS.
- Achieved caption H-glyph height at or above the profile minimum in all three.
- out/export-report.csv - dimensions, frames, duration, bytes, bits per second, size verdict.
- A stated crop-or-letterbox decision with a reason for each non-16:9 profile.

## Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

☐ out/export-report.csv in full.

☐ out/safe-area-check.csv showing every cue inside the safe rectangle.

☐ The computed font scale per profile and the achieved H-glyph height in pixels and as a percentage of frame height.

☐ The computed design-colour contrast ratio and a decoded-frame visual check.

☐ Your crop-versus-letterbox decision per profile, with the reason.

☐ One still frame from each rendition showing the caption and logo in place.

☐ The provenance statement confirming the exports inherit the source's SIMULATED label.

## Troubleshooting

| Symptom                                    | What to do                                                                                                                                                                                                                    |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Captions are cut off at the frame edge     | You positioned the text by a fixed pixel offset that does not scale with the profile. Compute the caption origin from the safe rectangle, never from a constant.                                                              |
| The logo has a black box around it         | You read the PNG without cv2.IMREAD_UNCHANGED, so the alpha channel was dropped. Read with four channels and blend using alpha.                                                                                               |
| cv2.putText renders nothing                | The font scale computed from getTextSize can come out near zero if you divided by the wrong element. getTextSize returns ((width, height), baseline) - the H-glyph height is the second element of the first tuple.           |
| The 9:16 export is mostly empty bars       | You letterboxed a 16:9 source into a 9:16 frame. That is arithmetically correct and usually the wrong choice - crop instead, and state the loss.                                                                              |
| Output duration is shorter than the source | Frames were dropped because a resized frame did not match the writer's declared size. Assert frame.shape[:2] == (out_h, out_w) before every write.                                                                            |
| File size fails the profile limit          | Record the failure and the lever you would pull. cv2.VideoWriter gives limited rate control; note that a production pipeline would hand off to a dedicated encoder, and say so rather than pretending the constraint was met. |

## References used in this lab

- [S11] Tschochohei and Schenker (2026), as above - \*Ch.7 pp.110-119\*.

Text-to-video, image-to-video and video-with-sound enterprise use cases; long-running operation polling pattern; prompt-driven per-second audio specification; prompt enhancement from a campaign brief.

- [S12] Tschochohei and Schenker (2026), as above - \*Ch.8 pp.127-132 and pp.140-143\*.

Modality-specific responsible-AI risks (stereotypes and representational harm, training-data and output ownership, deceptive imagery, voice cloning, deepfakes); provider safety comparison (SynthID watermarking and model cards; C2PA Content Credentials metadata; banned-word filters; configurable safety tolerance); transparency strategy - C2PA content credentials, robust invisible watermarking resilient to compression/cropping/format change, and model cards as a 'nutrition label for AI'.

- [S23] OpenCV 4.13 Python package, verified locally on the delivery image - \*cv2 4.13.0 (opencv-python 4.13.0.92)\*.

API-currency verification for every command taught: cv2.data.harcascades ships the frontal-face and eye cascades so no weights download is required; cv2.HOGDescriptor\_getDefaultPeopleDetector() is built in; SIFT\_create, ORB\_create, FastFeatureDetector\_create, cornerHarris, goodFeaturesToTrack, matchTemplate, createBackgroundSubtractorMOG2/KNN, meanShift, CamShift, calcOpticalFlowFarneback and PSNR are all present. IMPORTANT CORRECTION carried into the labs: cv2.TrackerCSRT\_create and cv2.TrackerKCF\_create are NOT in the base opencv-python wheel (they require opencv-contrib-python), and the cv2.quality SSIM module is also contrib-only - so the labs use cv2.TrackerMIL\_create and a NumPy SSIM implementation to stay dependency-light and runnable offline.

---

© 2026 Tertiary Infotech Academy Pte Ltd. UEN: 201200696W. Course material for TGS-2020505925, version v11.0.